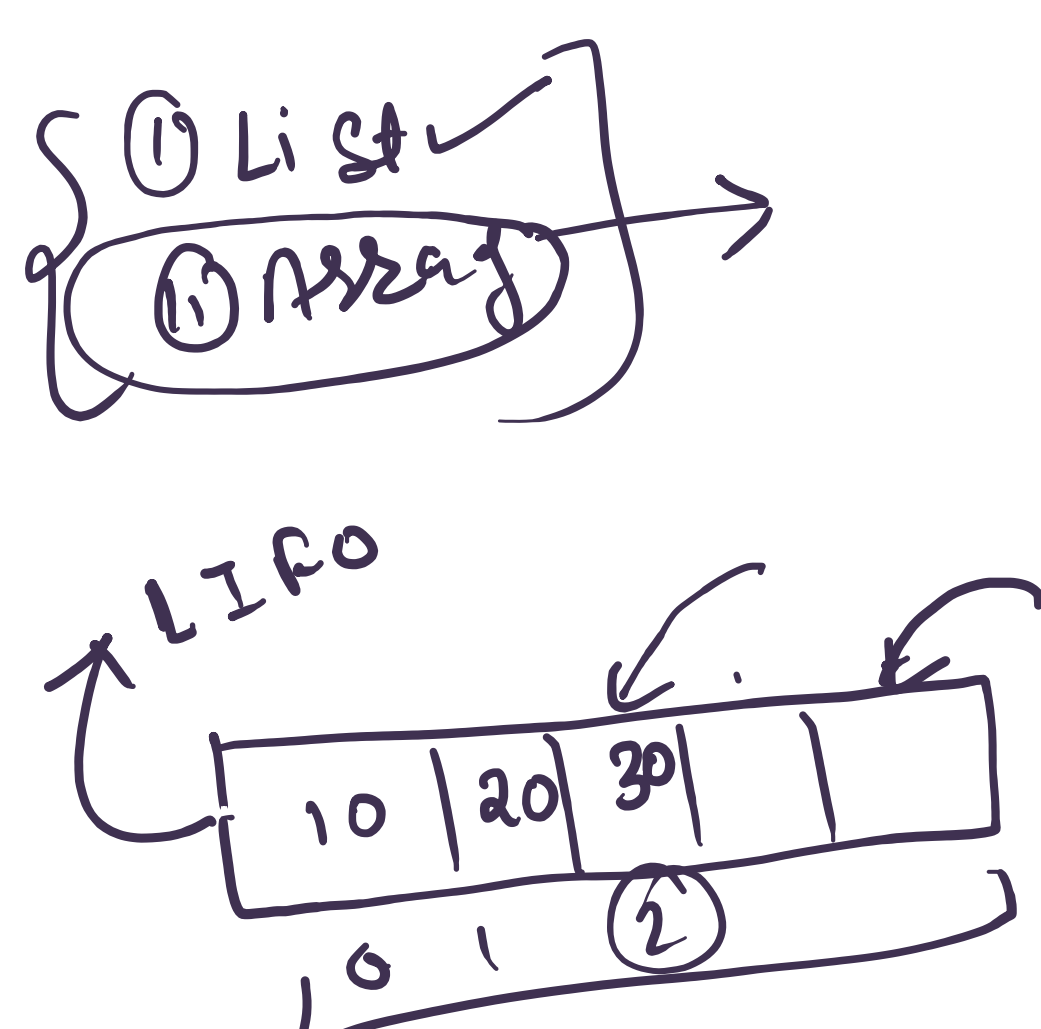
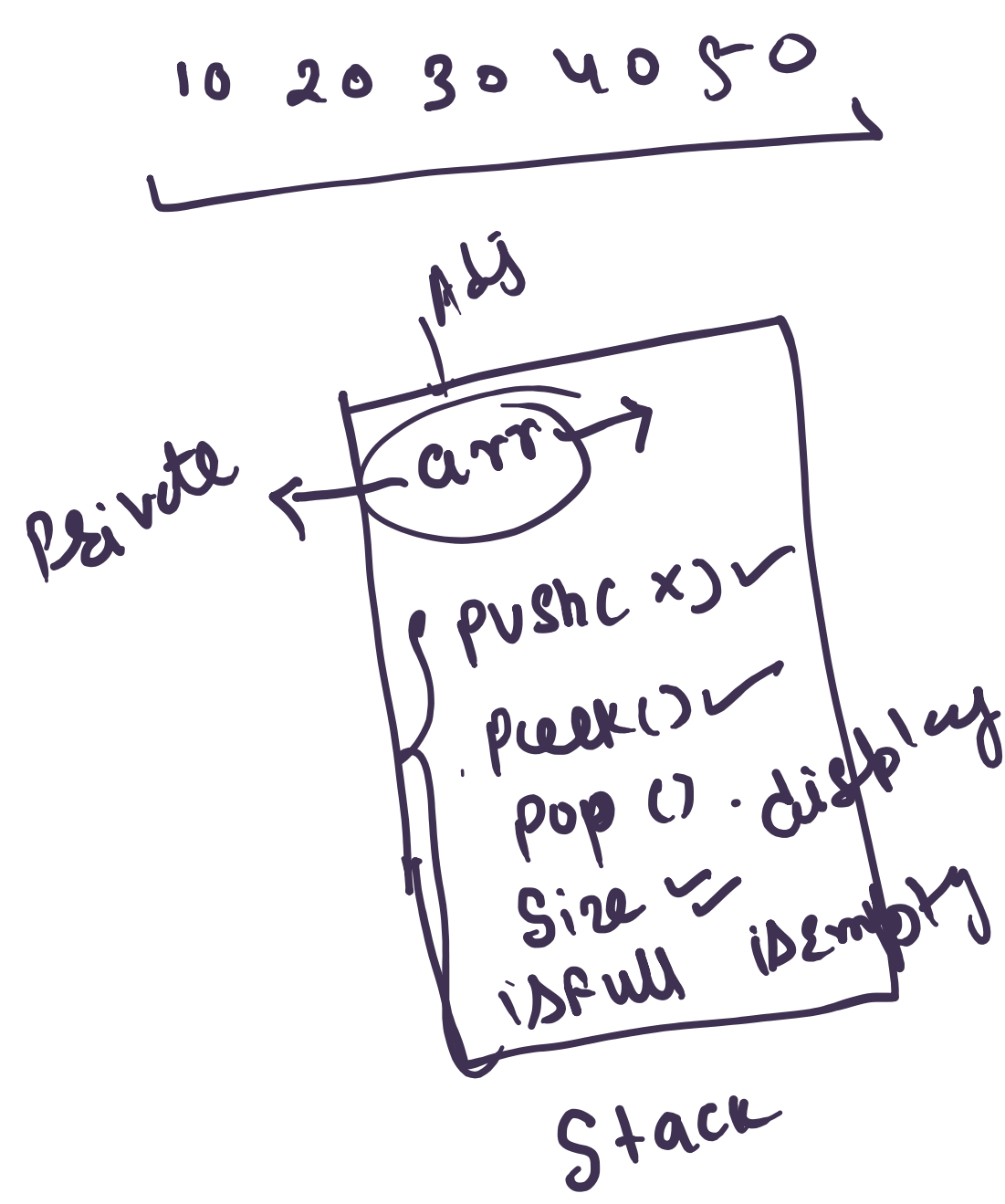
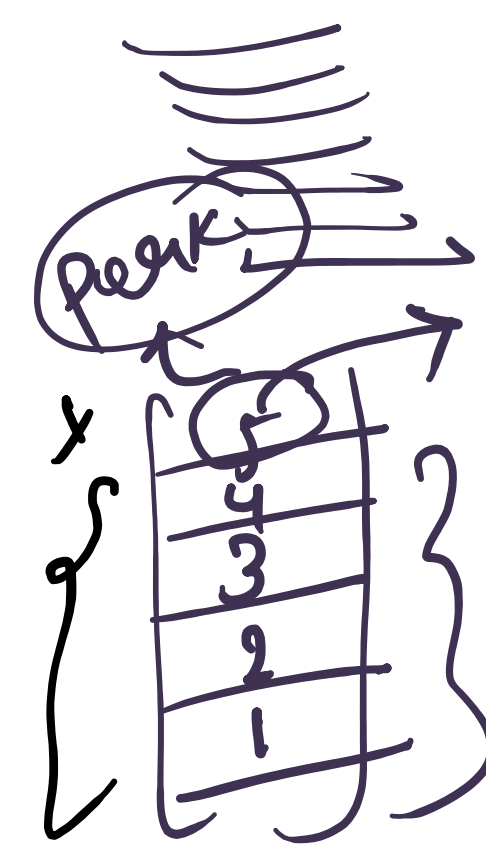
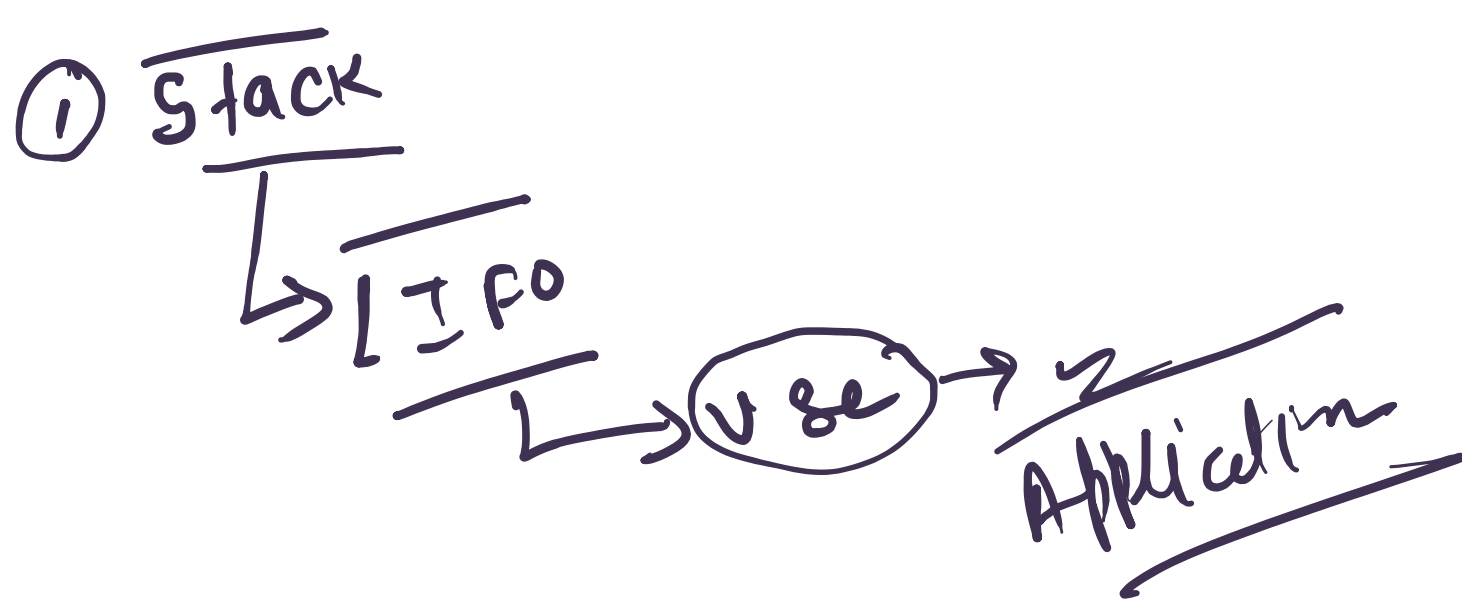
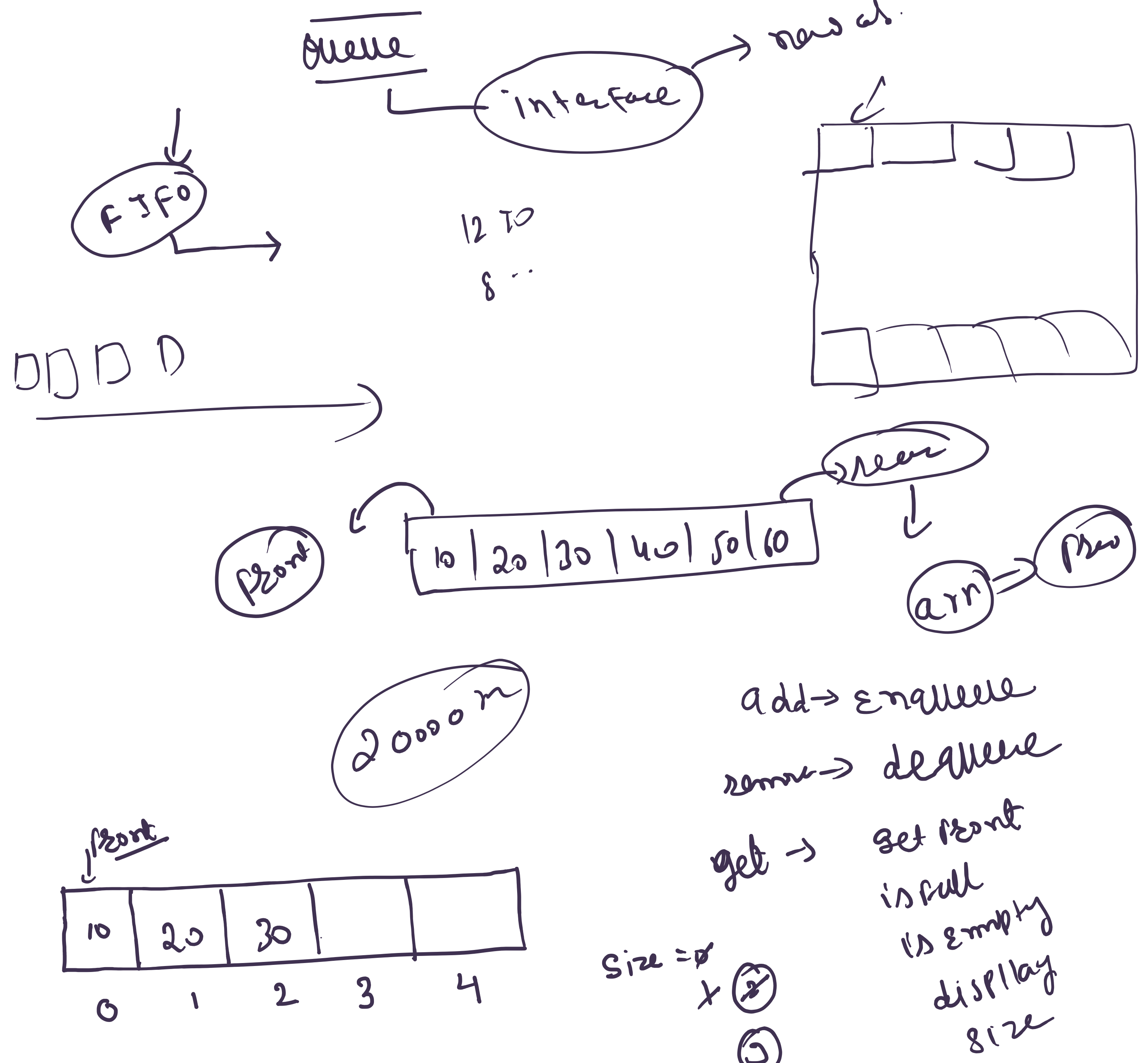
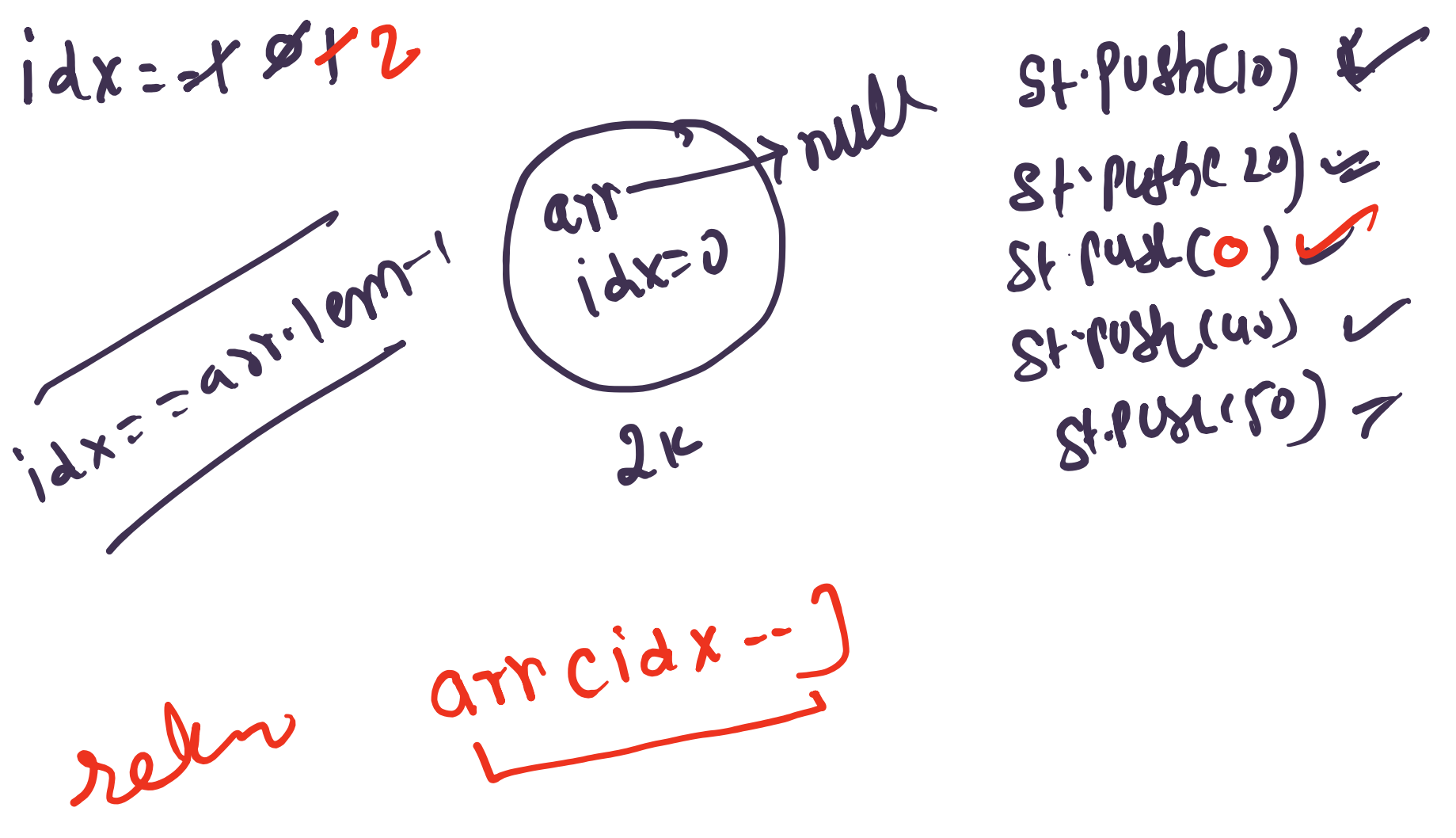
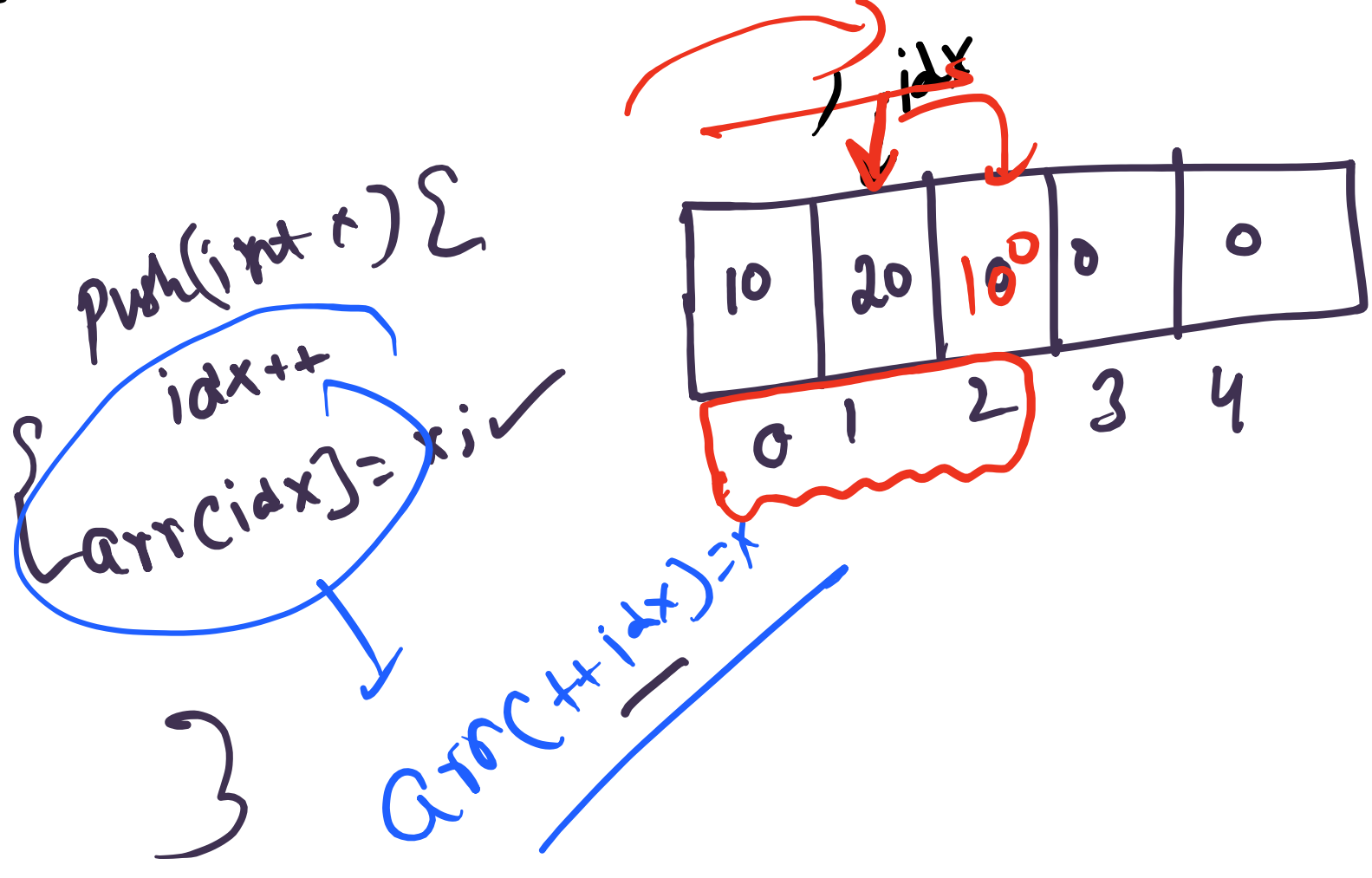


D.8 → Data ways



```
public class Stack {  
    private int[] arr;  
    private int idx;  
  
    public Stack() {  
        // TODO Auto-generated constructor stub  
    }  
    public Stack(int n) {  
        // TODO Auto-generated constructor stub  
    }  
}
```

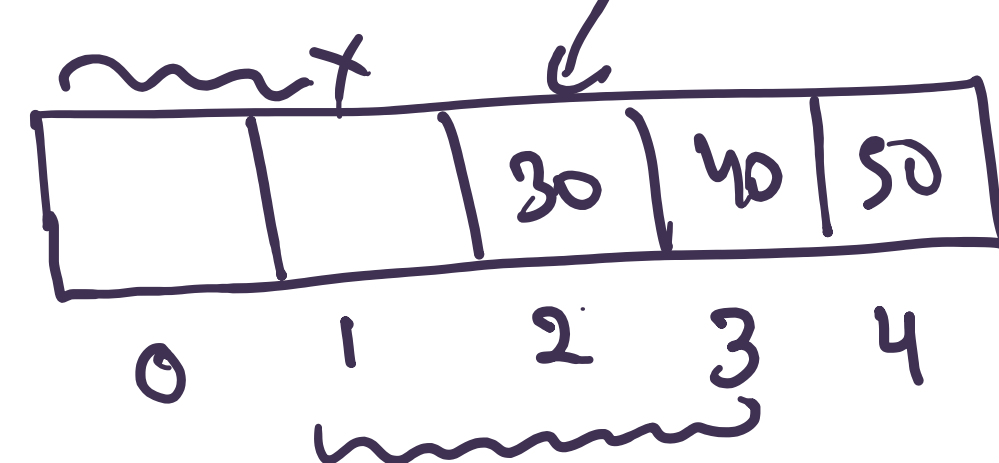
```
public class Stack_Client {  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        Stack st = new Stack(100);  
    }  
}
```



enqueue(int x) {
 arr[size] = x;
 size++;
}

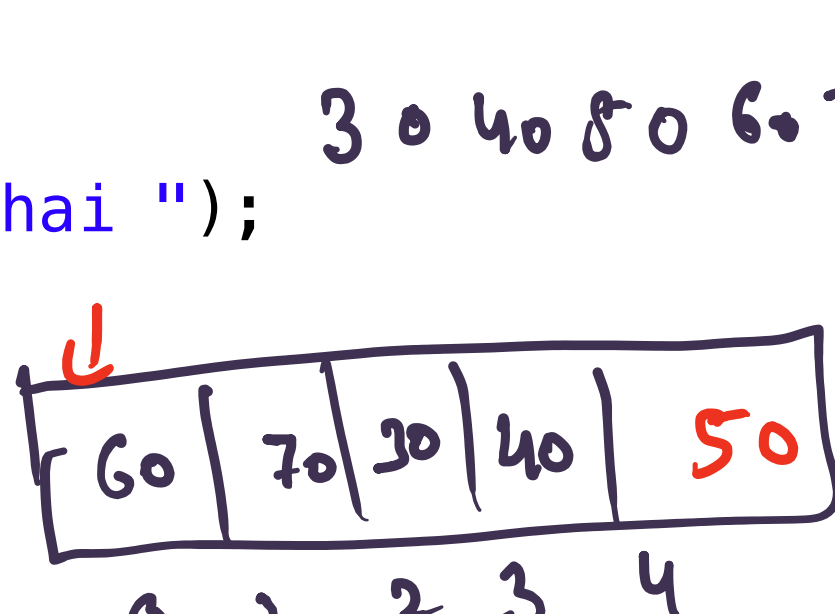
enqueue(10)
enqueue(20) ✓
enqueue(30) ✓

```
public boolean isEmpty() {  
    return size == 0;  
}  
  
public boolean isFull() {  
    return size == arr.length;  
}  
  
public int size() {  
    return size;  
}  
  
public void Enqueue(int x) throws Exception {  
    if (isFull()) {  
        throw new Exception("Bklol Queue full hai ");  
    }  
    arr[size] = x;  
    size++;  
}  
  
public int Dequeue() throws Exception {  
    if (isEmpty()) {  
        throw new Exception("Bklol Queue Empty hai ");  
    }  
    int x = arr[front];  
    front++;  
    size--;  
    return x;  
}
```

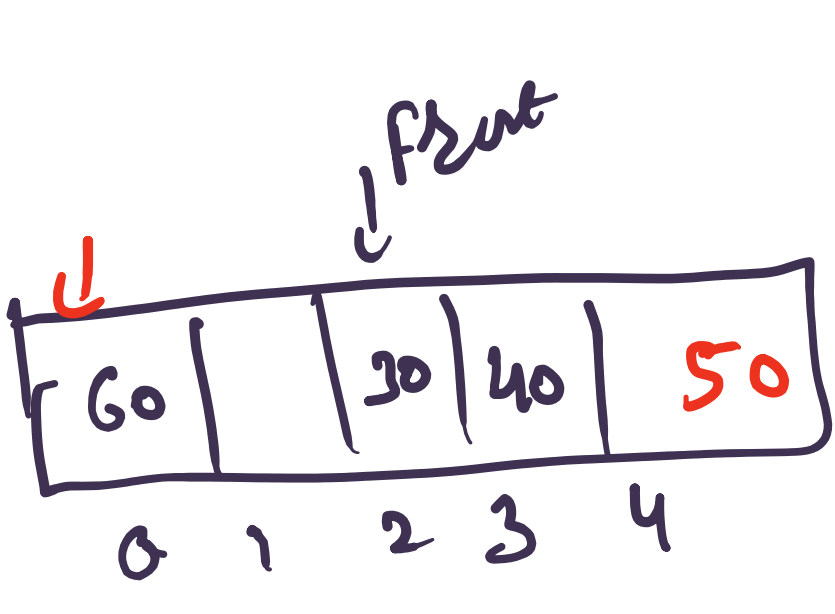


enqueue(10)
enqueue(20)
enqueue(30)
enqueue(40)
size = 4
front = 0
rear = 0

```
public void Enqueue(int x) throws Exception {  
    if (isFull()) {  
        throw new Exception("Bklol Queue full hai ");  
    }  
    int idx = front + size;  
    arr[idx] = x;  
    size++;  
}  
  
public int Dequeue() throws Exception {  
    if (isEmpty()) {  
        throw new Exception("Bklol Queue Empty hai ");  
    }  
    int x = arr[front];  
    front++;  
    size--;  
    return x;  
}  
  
public int getFront() throws Exception {  
    if (isEmpty()) {  
        throw new Exception("Bklol Queue Empty hai ");  
    }  
    return arr[front];  
}
```



enqueue(10) ✓
enqueue(20) ✓
enqueue(30) ✓
enqueue(40) ✓
size = 4
front = 0
rear = 0



size = 4

i = 0 front = 0 + 1 = 1 → 30 ✓
i = 1 front = 1 + 1 = 2 → 40 ✓
i = 2 front = 2 + 1 = 3 → 50 ✓
i = 3 front = 3 + 1 = 4 → 60 ✓

for (i = 0; i < size; i++) {
 int idx = (front + i) % arr.length;
 System.out.println(arr[idx]);
}